

BookML at the University of Edinburgh

Steven O'Hagan, Hayden Maudsley-Barton, and Charlotte Desvages

2025–26

This is a guidance document and demonstration of the template for HTML notes converted by BookML used by the School of Mathematics at the University of Edinburgh. The HTML version of this document can be found as a GitHub pages site. This GitHub repository contains the source LaTeX markup for the HTML site.

There is a blank template repository which should be used for your own course materials; see the documentation for more details on usage.

BookML is a small wrapper around LaTeXML, created and maintained by Vincenzo Mantova at the University of Leeds. The source code and local installation instructions are available on the BookML repository, and documentation is published at vlmantova.github.io/bookml

Contents

1	Conversion from L^AT_EX to HTML	1
1.1	Choosing your preferred workflow	1
1.2	Cloud-based workflow (via GitHub)	2
1.2.1	Step 1: Make a copy of the course notes template on GitHub . .	2
1.2.2	Step 2: Create an Overleaf project linked to your GitHub repository	4
1.2.3	Step 3: Write your notes in Overleaf and sync with GitHub . . .	4
1.2.4	Step 4: Download the generated HTML version of your notes from GitHub as a SCORM package, and upload it to Learn . . .	5
1.2.5	Step 5 (optional): Publish your notes as a public website	5
1.3	Local workflow	7
1.3.1	Using Docker	7
1.3.2	Installing BookML and dependencies manually	9
1.3.3	Sharing with students	10
1.3.4	Learning Technology Mailbox	10
1.4	FAQs	10
1.5	Other accessibility information	11
1.5.1	Other considerations	11
1.5.2	Alternatives to BookML	12
2	Usage and examples for the template	13
2.1	File structure	13
2.2	Including or excluding content from either version	14
2.3	Theorem styles	14
2.3.1	Solutions	15
2.4	Figures	16
2.4.1	Raster images	16
2.4.2	Vector images and diagrams	17
2.4.3	xy diagrams	17
2.5	Layout	19
2.5.1	Subfigures	19
2.5.2	Tables	20
2.5.3	Custom floats	20
2.6	Other	21
2.6.1	Multido	21

2.6.2	Monospaced output	21
2.6.3	Referencing from a bibliography	22
2.7	beamer slides (experimental)	22
3	Appendices	24
3.1	Context: why are we doing this?	24
3.1.1	Why are PDFs not “accessible”?	24
3.1.2	A solution: HTML with MathML	25
3.1.3	Insights from staff interviews	26
3.2	Flexibility is the key	27
3.3	Upload as ZIP	27

1 Conversion from $\text{\LaTeX}$ to HTML

1.1 Choosing your preferred workflow

There are several tools available to perform conversion of $\text{\LaTeX}$ to HTML. Here, we focus on BookML, which uses LaTeXML in the background; note that this is the same technology used by arXiv to produce HTML versions of research papers from the `.tex` source.

There are three steps to generating HTML notes from $\text{\LaTeX}$ using BookML:

1. write notes in $\text{\LaTeX}$, optionally using the School template;
2. use BookML to convert $\text{\LaTeX}$ code to HTML;
3. share the HTML version with students.

Your precise workflow will depend on how you wish to share the HTML version with students, whether you prefer to work in Overleaf or write $\text{\LaTeX}$ locally, and whether you prefer the HTML conversion to be dealt with in the cloud (via GitHub) or on your local machine (via Docker). Some possible workflows are summarised in the figure below.

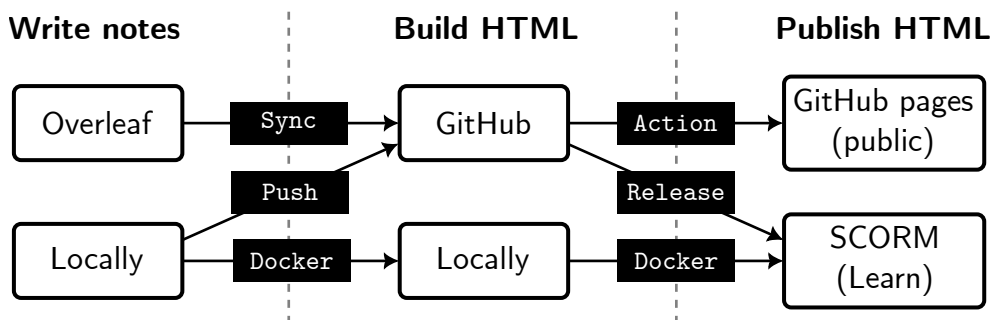


Figure 1.1: Workflow options to build HTML course materials.

We give detailed guidance for two of these workflows: one with cloud-based conversion and one with local conversion. You can mix-and-match or deviate from these documented workflows if you feel comfortable doing so. If you're not sure where to start, cloud-based is easiest (Section 1.2).

1.2 Cloud-based workflow (via GitHub)

The simplest way to generate HTML course materials makes use of GitHub to perform the conversion in the cloud, which means that you will not need to install anything on your computer. All you need to do is upload your $\text{\LaTeX}$ source code to a (private) GitHub repository; the conversion to HTML is then done automatically.

Additionally, if you use Overleaf, you can set it up to connect to GitHub. After the initial setup, there will be no need to interact directly with GitHub. Here, we explain this particular workflow¹. The main steps are as follows:

Step 1: Make a copy of the course notes template from GitHub.

Step 2: Create an Overleaf project, link it to your GitHub repository.

Step 3: Write your notes in Overleaf, and sync with GitHub.

Step 4: Download the generated HTML version of your notes from GitHub as a SCORM package, and upload it directly to Learn.

Step 5 (optional): Easily publish your notes as a public website using GitHub Pages.

In addition to the written instructions below, a video recording of a demo session showing the cloud-based workflow can be found on Media Hopper.

1.2.1 Step 1: Make a copy of the course notes template on GitHub

Create a GitHub account and join the UoE School of Mathematics organization

If you already have a GitHub account, then please email the Learning Technology Team lt@maths.ed.ac.uk giving your GitHub username, and ask to be added to the UoE School of Mathematics GitHub organization.

If you do not have a GitHub account, then please email the Learning Technology Team lt@maths.ed.ac.uk and ask for an invitation to the UoE School of Mathematics GitHub organization. You will receive an email from GitHub with a link to create an account and join the SoM organization. Please note that the email may not render properly in your mail client – the link may appear as an empty box but this is the invitation link!

This invitation will expire in 7 days.



Figure 1.2: GitHub invitation link when it appears empty.

¹You should also follow this section if you don't plan to use Overleaf, but still want to use the cloud-based conversion to HTML. The instructions will indicate where to deviate from these steps.

Make a new GitHub repository for your notes using the SoM template

Sign in to your GitHub account, and go to the blank template repository. Then, click the “Use this template” button at the top right of the page. This will create a new repository for you containing the template files.

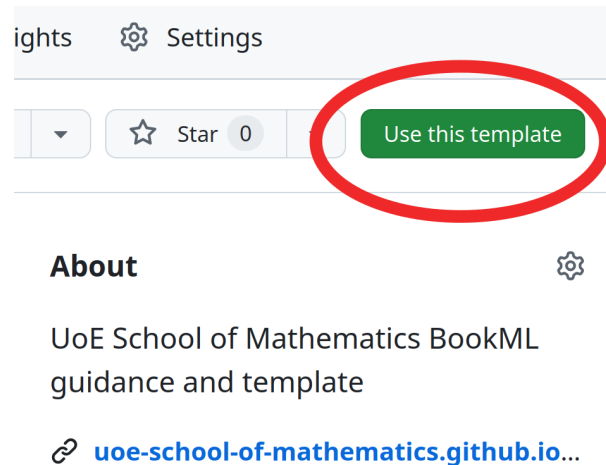


Figure 1.3: “Use this template” button is at the top right of the page.

The “Create a new repository” page has some options:

- Keep “Include all branches” **unticked**.
- Change “Owner” to **“UoE-School-of-Mathematics”**. This will ensure that your notes are stored in a private repository on GitHub in the UoE School of Mathematics organization².
- Choose an appropriate repository name; we recommend that you choose your **course abbreviation** (e.g. FAC, IMU, IDS, FPM...), or include it in the name.
- Make sure “Visibility” is **“Private”**.

Finally, click “Create repository” and wait at least 3 minutes before moving on. GitHub needs some time to initialise the project.

Not using Overleaf?

At this stage, you will have your own copy of the SoM template, as your own repository on GitHub. Steps 2 and 3 below use Overleaf, which makes things easier if you’re not familiar with GitHub. However, it’s also possible to not use Overleaf at all if you prefer to work locally:

- If you are comfortable working with git, you can clone your new repository, and push your changes to GitHub to update the HTML version.

²If you prefer the repository to be owned by your own GitHub account instead, one important difference is that the GitHub Actions minutes needed to perform the HTML conversion in the cloud will be taken from your own GitHub account’s quota. The SoM organization has a larger allowance; this is why we encourage you to use it.

- If not, you can also download the files from GitHub (following these instructions), extract the .zip, write your notes in that folder, and re-upload any .tex files you have changed directly to GitHub (following these instructions) every time you want to update the HTML version.

In any case, if you do this, you can skip directly to section 1.2.4.

1.2.2 Step 2: Create an Overleaf project linked to your GitHub repository

Link your Overleaf account to your GitHub account

Visit your Overleaf account settings. Under **Project Synchronisation**, click **Link** next to **GitHub Sync**. Follow the prompts to link Overleaf and GitHub.

Make an Overleaf project from your GitHub repository

At this stage, you should have a GitHub repository created from the SoM template. In Overleaf, create a new project, ensuring you choose **Import from GitHub**. Then, choose **Import to Overleaf** for the repository you created previously.

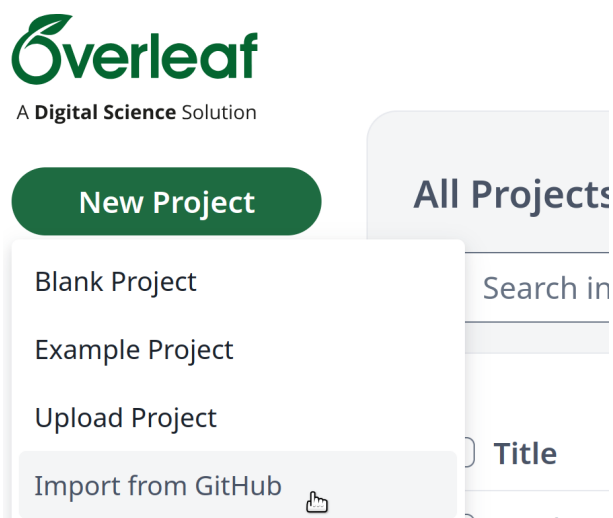


Figure 1.4: Choose “Import from GitHub” when creating the new project.

Once this is complete, you should have a new Overleaf project containing the template files.

1.2.3 Step 3: Write your notes in Overleaf and sync with GitHub

Now you can work on your notes just like with any other Overleaf project.

Whenever you want to publish changes to the HTML version, you need to push the changes to your GitHub repository. To do this, in Overleaf, click **Menu** (top left) and then choose **GitHub** under the **Sync** menu.

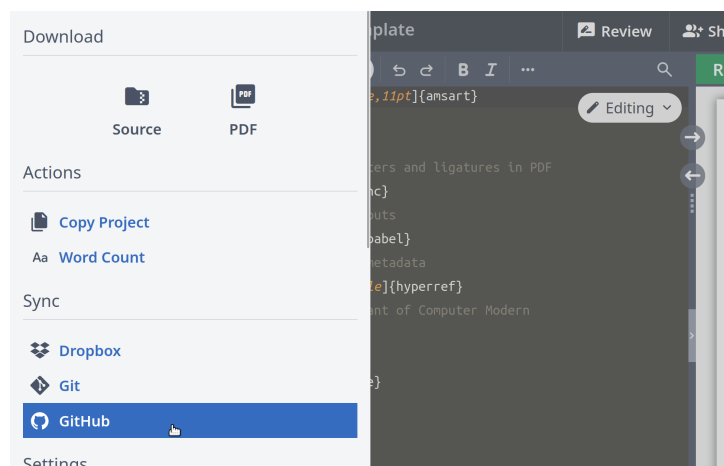


Figure 1.5: The “GitHub” option in the Overleaf top-left menu.

Click **Push Overleaf changes to GitHub**, enter a description of the changes you have made (optional) and then click **Sync**.

1.2.4 Step 4: Download the generated HTML version of your notes from GitHub as a SCORM package, and upload it to Learn

Every time you sync new changes to GitHub, the HTML notes will recompile automatically (this takes a few minutes). Visit your GitHub repository; a green check mark above your files indicates that this has completed. Scroll to “Releases” in the right-side menu, and click on the `SCORM.main.zip` file³. This will download a zip file containing the HTML version of your notes.

Upload the SCORM file

- On your Learn page, create a new content item.
- Choose the “SCORM package” option.
- Upload the file `SCORM.main.zip`.
- Once it’s uploaded, untick the “Mark SCORM” checkbox, and click “Save”.

1.2.5 Step 5 (optional): Publish your notes as a public website

Additionally, or alternatively, you can publish your notes as a **public website** using GitHub Pages. The advantage of doing this, is that this is all already automated; so there is no need for you to manually update your published site every time you update your notes (as you need to do when uploading to Learn). Instead, you can simply add a link

³The word `main` may be replaced by the name of any `.tex` file with `documentclass` in it. If you have multiple separate documents in your project, you will get multiple SCORM files to upload on Learn separately.

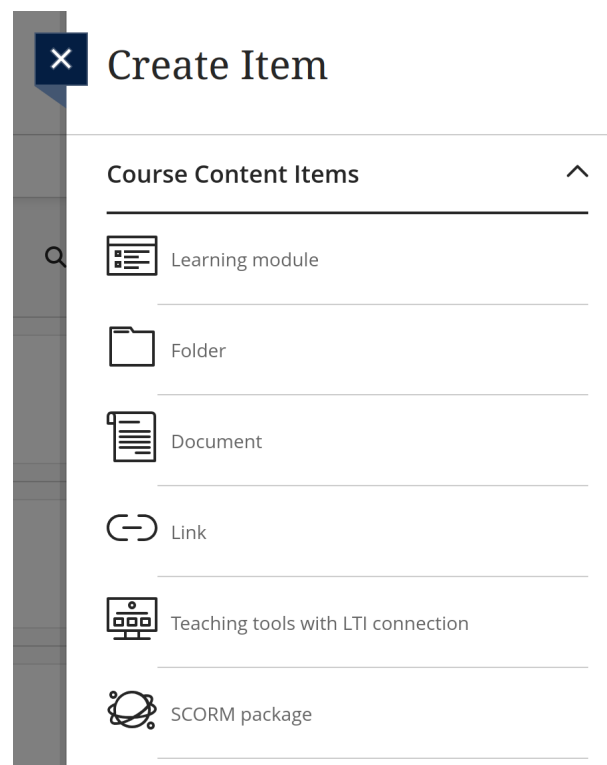


Figure 1.6: The “SCORM package” option on Learn Ultra.

to your published website on your Learn page, and this will always point to the most up-to-date version of your notes.

Your site is already built automatically by using the GitHub template, but it’s not published by default. To set up a publicly-accessible website with your HTML notes, first visit your repository on GitHub. In the top menu, click **Settings**, then **Pages** in the left menu.

Under **Build and deployment**, look for the **Branch** option. From the **Select branch** option, choose `gh-pages` and **Select folder** `/docs`.

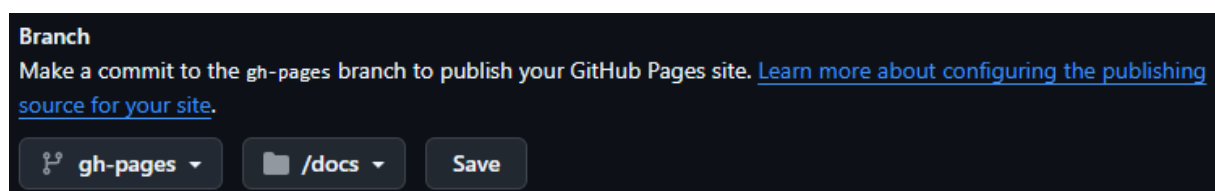


Figure 1.7: Select the `gh-pages` branch and the `/docs` folder.

Wait 1 minute then refresh the page. You should now see a link to the publicly-available site hosting your HTML notes. This includes the URL that you should use to share your notes with students.

Behind the scenes, GitHub will now automatically rebuild your HTML notes and update the public website every time you synchronise changes with GitHub. This was a one-

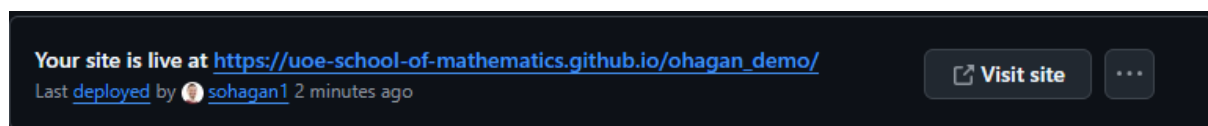


Figure 1.8: The link to your course site.

time setup; you don't need to interact directly with GitHub again.

Copyright and license

As per UK law, as explained in University guidance, the University “is the first owner of the rights of any work made in the course of your employment”, and this applies to teaching materials. By default, this is attributed at the bottom of the index page of your published site.

The University also has an Open Educational Resources policy and guidance; we encourage you to consider publishing your course materials as an open educational resource (and change the copyright notice accordingly).

1.3 Local workflow

Another way to create and maintain HTML course materials is to convert your notes locally on your machine. To do this, you will need to either install Docker, or directly install BookML and its dependencies. We give detailed instructions for the Docker workflow, as this is generally easier to set up and use.

1.3.1 Using Docker

To run BookML, we can use a Docker container — a lightweight, portable unit that packages an application and all its dependencies, ensuring that it runs consistently across different environments. This removes the need to download software and maintain dependencies on your computer.

Importantly for us: there is a Docker image available for BookML, which is ready-to-use and contains everything we need.

To use it, we can use Docker Desktop, which is available for Windows and Mac, or Docker Engine, which is available for Linux. An open-source alternative (compatible with Docker) is Podman, which is available for Windows, Mac, and Linux. In any case, the key steps are:

1. Setup (just once): install Docker or Podman, and download the Docker image.
2. Convert: run a single command in the folder with your `.tex` files.

Both Docker and Podman allow us to use either the command line or a graphical interface. If you don't know which one to choose, just pick one and try it!

Notes:

- If you have a managed Windows machine, Docker Desktop is available from the software centre. This is a slightly complicated process, see Appendix ?? for instructions. It may be simpler to request ‘Make Me Admin’ and install from the internet. If you wish to use Podman, you will need to request ‘Make Me Admin’. Guidance on ‘Make Me Admin’.
- The Learning Technology team will be able to assist you with installing Docker or Podman — email lt@maths.ed.ac.uk to ask for help.

Setup (first-time only)**Option 1 - to set up Docker:**

1.
 - **Windows/MacOS/Linux:** Download Docker Desktop from **this** link or the Software Centre, and install as instructed by the interface. Launch Docker Desktop. There are versions of the download file of Docker Desktop which cause an error. Ensure you are downloading the version that is listed in the documentation.
 - **Alternatively for Linux only:** Download and install Docker Engine, following the instructions for your distribution.
2. Launch a terminal (use `cmd.exe` on Windows), and run the command:
`docker pull ghcr.io/vlmantova/bookml:latest`
This will download the Docker image to your machine.

Option 2 - to set up Podman: Note that this requires “Make Me Admin” on Windows managed machines.

1. Download Podman Desktop from [here](#) and install as instructed by the interface.
2. Podman desktop will open, and you will be asked to setup. Click the ‘Setup’ button and follow through the instructions that follow. When prompted, select ‘Yes’, and then ‘Install’.
3. Once fully installed, we can download the image. To do this, open the ‘Images’ tab (the 4th option on the left-hand side of the screen) and click Pull on the top right of the screen, then enter `ghcr.io/vlmantova/bookml:latest` and click “Pull image”.

Conversion

Before you start, for better results, add `\usepackage{bookml/bookml}` to the preamble of your `.tex` file(s) (after the `\documentclass{...}` command). This is not necessary if you are using the template provided here by the School of Mathematics (download and extract “Source code (zip)”).

Note that if you use Overleaf, the `bookml/bookml` package will cause an error, unless you also add the `bookml` folder to your project (found in `release.zip` on the BookML repository).

Option (a) - using the command line:

1. Start a terminal in the directory containing your `.tex` file(s). (Use `cmd.exe` in Windows.)
2. Run the command:

```
{software} run -t -v ./source ghcr.io/vlmantova/bookml:latest
```

(replace `{software}` with `docker` or `podman`).

Option (b) - using the graphical interface:

1. In Docker/Podman Desktop, click on the 'Images' tab, and you will see the image you have just downloaded. Click the 'Play' button (Figure 1.9).
2. Under "Volumes", specify the "Host path" to be the path to your folder, and the "Container path" to be `/source`. (In Docker Desktop, this is under "Optional settings" – see Figure 1.10.)
3. Click 'Run' (Docker) or 'Start Container' (Podman) to produce the PDF and HTML outputs.

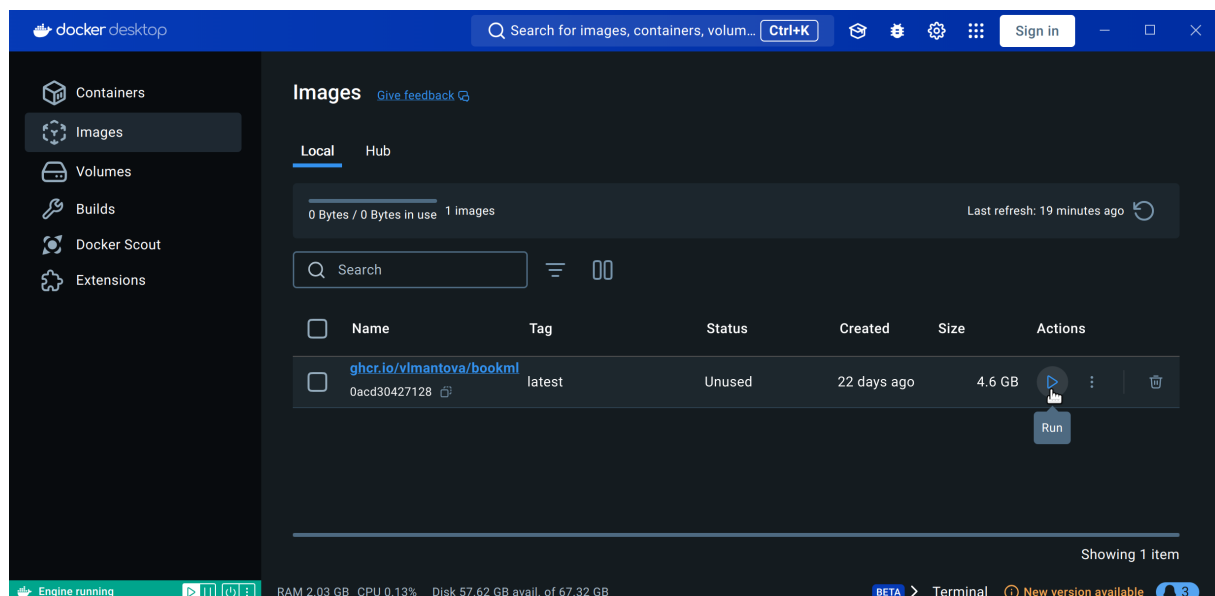


Figure 1.9: Step 1: click the 'play' button to run the command.

1.3.2 Installing BookML and dependencies manually

If you prefer to install the necessary packages and dependencies to your machine instead of using Docker, instructions are given in the BookML documentation for the installation and the conversion to HTML.

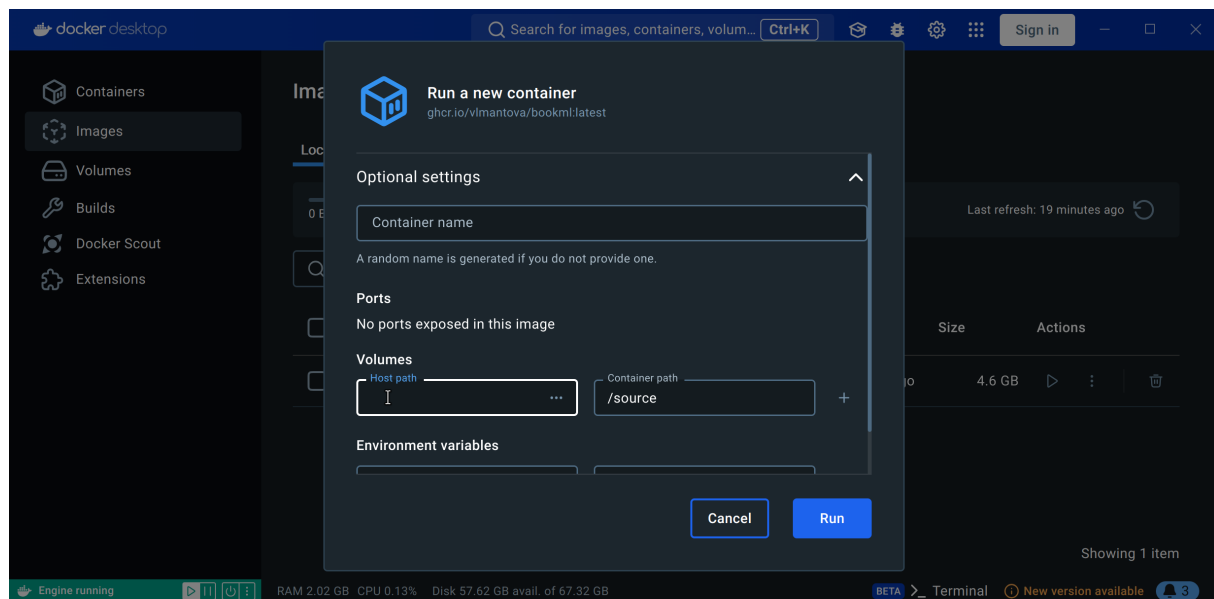


Figure 1.10: Step 2: give Docker the path to your folder.

1.3.3 Sharing with students

Running BookML will produce PDF files (with `pdflatex`), as well as `.zip` files containing the HTML versions of your `.tex` files, which can be uploaded directly to Learn. The output (both PDF and HTML) will be found in the same folder as your `.tex` files. Upload `SCORM.yourfile.zip` directly to Learn as described in 1.2.4.

The `yourfile.zip` (without the `SCORM`) files contain a standalone HTML site, which you may also wish to publish yourself if you are comfortable doing so (e.g. on your SoM user domain). See 1.2.5 for related copyright information. However, only the `SCORM` versions are fully compatible with Learn, so please use these if you plan to upload your materials to Learn.

1.3.4 Learning Technology Mailbox

If you would rather, you can also email `lt@maths.ed.ac.uk` with a `.zip` of the folder containing all the source files for your course materials (everything needed for your `.tex` files to compile normally). The Learning Technology team will perform the conversion, and email you back with a `.zip` file containing the HTML version of your course materials; optionally, they can also upload it to your Learn page. If they spot anything which didn't convert well, they will let you know.

1.4 FAQs

Q: Error: Warning: no `.tex` files with `\documentclass` found in this directory

A: The software cannot reach your files, this may be due to running the software in the wrong place, permissions, or a remote directory. Double check your file

path, and try moving the files to a local directory and try again.

Q: Why aren't my files converting at all, there is not an error, BookML simply states there are no files to convert.

A: This is likely due to the fact that the software cannot reach your files. This can be because your files are in a folder, or that your files have forbidden file characters or a space.

Q: The conversion process is taking a very long time.

A: The conversion process can take a while, especially for larger documents — but if it's 30 minutes or more, then please flag this. Note that, if you are using a local installation (not Docker), there are some known issues related to this:

- There is a known issue with the package `exp13` when used with TeXlive 2022 and older which can cause the conversion to take longer.
- There is a known issue with the “postprocessing XSLT” stage taking several minutes (instead of seconds) on Ubuntu 22.04.

Q: Will my .tex files be converted if I comment out `\documentclass`?

A: Yes, and it may cause errors. For files which you don't want to be converted standalone (e.g. files which are `\input`ted into another file), delete the `\documentclass` command entirely or move them into a folder.

1.5 Other accessibility information

This is not the whole picture. There are other things to consider when creating accessible documents that is not handled by materials being in HTML. Additionally, there are other alternatives to a $\text{\LaTeX}$ to HTML conversion.

1.5.1 Other considerations

- **Images:** Ensure that all images have *quality* alternative text. This is not done automatically by BookML, so you will need to add this manually. Information on how to add alt text is in the demo portion of this document (see Section 2.4), with information about images.
- **Links:** Ensure that all link text is accessible.
- **Use of colour:** Colour should not be used exclusively to communicate information. This is often missed in graphs and charts. For example, portions of a pie chart should be directly labelled, or the legend should be additionally texture-based.

1.5.2 Alternatives to BookML

If you are not strongly attached to using $\text{\LaTeX}$, then instead of using BookML, you can create course materials in accessible formats using the Markdown language. Software such as Bookdown or Quarto offer Markdown support and extended features, and allow you to produce materials in a range of formats (including PDF, HTML, computational notebooks. . .) directly from Markdown source.

2 Usage and examples for the template

The School of Mathematics provides a default template, which is designed for writing new course materials – it aims to improve accessibility and convert nicely to HTML. It should also be relatively easy to convert existing materials to this template. The template provides a preamble in `preamble/SoM.tex`. You can add your own preamble in `preamble/custom.tex`.

The examples shown in this section are intended to demonstrate the use of the packages and environments included by default in the template, and to give an overview of how different elements are rendered in both the PDF and the HTML versions of a document. Workarounds and $\text{\LaTeX}$ snippets are also given for certain packages or commands which are not compatible with BookML; these may also be useful to those not using the template.

For more information, see the BookML Documentation. This also includes interesting ways of using the HTML functionality.

2.1 File structure

All content should go either in `main.tex` (single file), or in separate `.tex` files located in the same folder as `main.tex`. If using separate sub-files, they should be included into `main.tex` using `\input{}`, as is done in this document. Sub-files should **not** have `\begin{document}` or `\end{document}` commands.

The `main.tex` template uses the `book` document class; therefore, the levels of sectioning available are `\part`, `\chapter`, `\section`, `\subsection`, `\subsubsection`, `\section*`, `\chapter*`, `\section*`, `\subsection*`, `\subsubsection*`. If you prefer to use `article` as the document class, then you should use the code in `main-article.tex` instead.

You can use `preamble/custom.tex` to complement the template preamble with any additional packages, macros, custom environments, and so on.

2.2 Including or excluding content from either version

The `bookml` $\text{\LaTeX}$ package includes the command `\iflatexml` to include or exclude particular code from either the PDF or the HTML version of your notes.

- Use `\iflatexml` to delimit anything (content, packages, macros, ...) that should only be used in the HTML version.
- Use `\iflatexml\else` to delimit anything (content, packages, macros, ...) that should only be used in the PDF notes.

You can check the following example in the source file for this document:

This will only show up in the PDF version.

$\text{\LaTeX}$ snippet

This will only show up in the PDF version.

2.3 Theorem styles

`preamble/SoM.tex` defines a range of theorem-like styles, and sets up sequential numbering indexed on `chapter`. Here is an example of a `theorem` and a `proof` environment. Theorem 2.1 and its proof will be in the exam:

Theorem 2.1. Let $\mathbf{e}_k = \mathbf{R}^k \mathbf{e}_0$, for $k \in \mathbb{N}$ and some $\mathbf{R} \in \mathbb{R}^{n \times n}$. If $\|\mathbf{R}\|_p < 1$, then $\|\mathbf{e}_k\|_p \rightarrow 0$ as $k \rightarrow \infty$. Change

Proof. We have

$$\begin{aligned} \|\mathbf{e}_k\|_p &= \|\mathbf{R}^k \mathbf{e}_0\|_p \\ &\leq \|\mathbf{R}^k\|_p \|\mathbf{e}_0\|_p \\ &\leq \|\mathbf{R}\|_p^k \|\mathbf{e}_0\|_p \end{aligned}$$

and so $\|\mathbf{e}_k\|_p \rightarrow 0$ as $k \rightarrow \infty$ if $\|\mathbf{R}\|_p < 1$. □

Note that the `align` environment is used in the proof, and also works as expected in the HTML version.

There is also a numbered “Example” environment, which has a left-side bar to differentiate it from the rest of the notes. To remove the numbering, see `SoM.tex`.

Example 2.2. Let

$$\mathbf{A} = \begin{bmatrix} 2 & 1 & 1 \\ 4 & 3 & 3 \\ 8 & 7 & 9 \end{bmatrix}.$$

We want to put $\mathbf{A}$ into row echelon form. We create zeros below the diagonal in the first column by subtracting multiples of the first row from the other rows:

$$\mathbf{L}_1 \mathbf{A} = \begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ -4 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 2 & 1 & 1 \\ 4 & 3 & 3 \\ 8 & 7 & 9 \end{bmatrix} = \begin{bmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 3 & 5 \end{bmatrix}.$$

Repeating this for the second column

$$\mathbf{L}_2 (\mathbf{L}_1 \mathbf{A}) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -3 & 1 \end{bmatrix} \cdot \begin{bmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 3 & 5 \end{bmatrix} = \begin{bmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 2 \end{bmatrix} = \mathbf{U}.$$

The above procedure factorises $\mathbf{A}$ as $\mathbf{A} = (\mathbf{L}_2 \mathbf{L}_1)^{-1} \mathbf{U}$.

And some extra text after the example to show the return to the regular layout.

If you wish to use a different theorem style, you can do so by editing `preamble/SoM.tex`.

To reflect these changes in the HTML, you can edit the CSS in `bmluser/SoM.css`.

`.ltx_theorem_theorem` is the class for theorems, and `.ltx_theorem_proof` and similar. to edit specifically wish to edit the way it looks in sepia mode or night mode. Add css to `.color-theme-1 .ltx_theorem_theorem` and `.color-theme-1 .ltx_theorem_theorem` respectively.

2.3.1 Solutions

If you wish to hide create a version of the documents with and without solutions, this is generally done using the `version` package. This is not compatible with BookML. However, we can navigate this with a few options.

The first choice is to replicate the package using the `comment` package. Add this code to your preamble.

```
\iflxml
  \usepackage{comment}
  % \excludecomment{sol}
  \includecomment{sol}
\else
  \usepackage{version}
  % \excludeversion{sol}
  \includeversion{sol}
\fi
```

Alternatively, if you wish to allow students to view solutions but have them hidden, you can use HTML to create a dropdown that students can access with the solution under. Uncomment the following code in `SoM.tex`.

```
\iflxml
  \renewenvironment{solution}
```

```

    {\<details style="text-align: left; width: 100\%" open="">
    \<summary> \textbf{Solution} \</summary> \}
    {\</details> }
\else
  \usepackage{version}
  % \excludeversion{solution}
  \includeversion{solution}
\fi

```

This would look like:

Solution. This is how we solve a quadratic.

2.4 Figures

2.4.1 Raster images

Figure 2.1 is a PNG image included with `\includegraphics`. The `\img` macro is set in `preamble/SoM.tex`, which you can change there if your graphics folder has a different name.

Use the command `\alttext{alternative text}` in a figure environment to provide **informative alternative text** for all your images — this is an **important accessibility feature**. Here is useful advice on writing good alternative text.

To avoid images being inverted in night mode in HTML, add `\bmlPlusClass{bml_no_invert}` after the `\includegraphics` command. There should be no linebreaks or spaces between. This can also be used for tikz images.



Figure 2.1: A PNG image included with `\includegraphics`.

2.4.2 Vector images and diagrams

`tikz` **images**

Figure 2.2 is a `tikz` diagram included with `\input`.

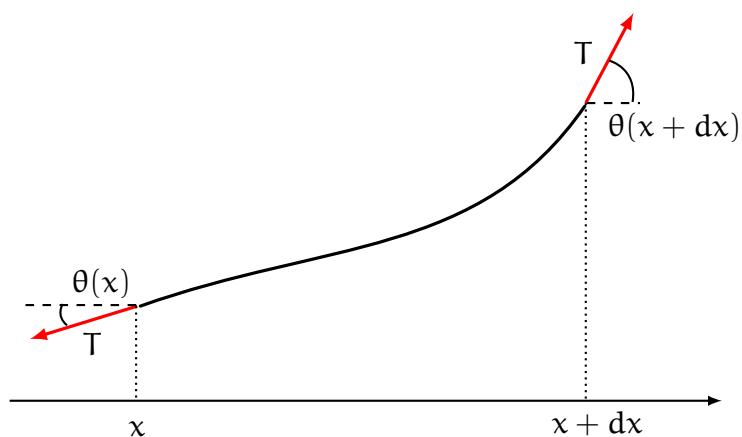


Figure 2.2: Small string element under tension.

2.4.3 `xy` diagrams

Figure ?? shows an example of a diagram, reproduced from the BookML documentation.

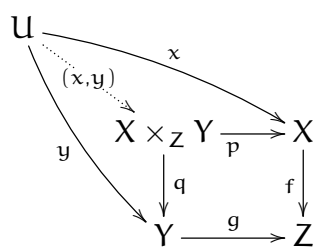


Figure 2.3: An example of a xymatrix.

pgfplots

PGF plots are also supported, as shown in Figure 2.4.

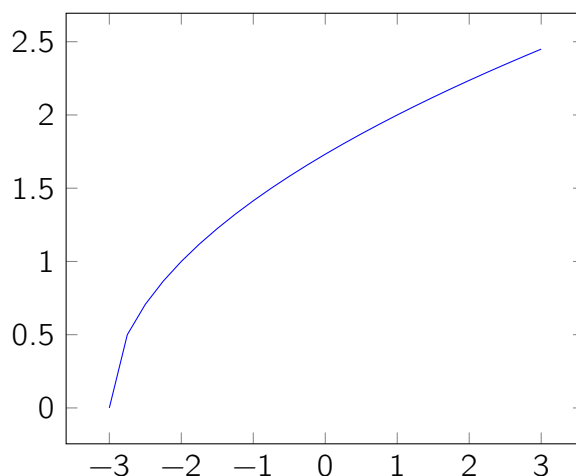


Figure 2.4: Plot created using the pgfplots package.

SVG

Unlike $\text{\LaTeX}$ HTML can render SVG images directly. If you have SVG images or diagrams as source files, this will produce a clearer image in the HTML version (for example, TikZ images are rendered as SVG). This can be done by using `\iflaxml` to include the SVG image in the HTML version, and the PDF image in the PDF version. This is done below. More information on how this works can be seen in Section 2.2.



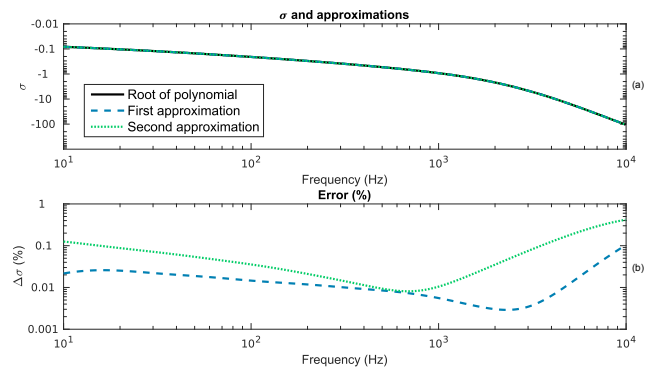
2.5 Layout

2.5.1 Subfigures

The subfigure environment from the subcaption package can be used for both the PDF and HTML versions, as seen in Figure 2.5. Alternatively, the minipage package can be used, as shown in Figure 2.8. Unlike subcaption, minipage forces the figures to be side by side, even if they are too wide to fit on the page. This may be useful if you want to force the figures side by side, but you may wish to swap to subcaption for best flow.



(a) A JPEG image as a subfigure.



(b) An EPS file as a subfigure.

Figure 2.5: Two subfigures, using the subcaption package.



Figure 2.6: A JPEG image as a subfigure.

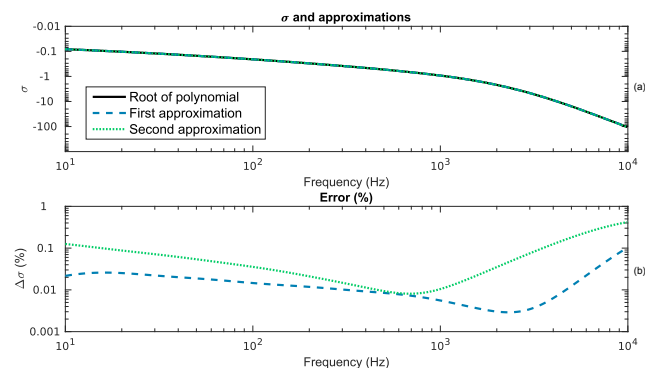


Figure 2.7: An EPS file as a subfigure.

Figure 2.8: Two subfigures, using the minipage environment.

Wrapping text around figures

cell1 dummy text dummy text dummy text	cell2	cell3
cell1 dummy text dummy text dummy text	cell5	cell6
cell7	cell8	cell9

Table 2.1: Example table produced with the `tabular` environment.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Figure 2.9: Text wrapped around an image using `wrapfigure`.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

2.5.2 Tables

Tables can be typeset using the `tabular` environment (see Table 2.1). Note that the `tabularx` package is also compatible.

2.5.3 Custom floats

New float types can be created with the `float` package. For example, Program 1 is a custom `program` float (adapted example from the LaTeX Wikibooks). The code is displayed differently but the conversion works.

The package `multicol` is compatible with BookML, but the in the HTML version, there will not be multiple columns. See below, in the PDF version, there are three columns, but in the HTML version, it will be a single column.

Program 1 The Hello World! program in Java.

```
class HelloWorldApp {
    public static void main(String[] args) {
        //Display the string
        System.out.println("Hello World!");
    }
}
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tor-

tor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor

lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

2.6 Other

2.6.1 Multido

The `multido` package is compatible with BookML. It can be used to create loops in your code. For example, the following code will produce a sequence of 8 numbers from 2.00, with a step size of -3.05:

2.00, -1.05, -4.10, -7.15, -10.20, -13.25, -16.30, -19.35,

2.6.2 Monospaced output

The `listings` package is compatible with BookML. It can be used to create code listings in your document. For example, the following code will produce a listing of a simple Java program:

```
class HelloWorldApp {
```

```
public static void main(String[] args) {  
    //Display the string  
    System.out.println("Hello World!");  
}  
}
```

The `verbatim` environment is also compatible with BookML. It can be used to create verbatim text in your document. For example, the following code will produce a verbatim block of text:

This is a verbatim block of text.
It will be displayed exactly as it is
written, with no formatting or special characters such as `\LaTeX`

2.6.3 Referencing from a bibliography

The package `natbib` is compatible with BookML. Note that `biblatex` is not yet compatible.

This is an example reference [Str04], and another [PFCC16]. It should not be complex to switch from `biblatex` to `natbib`.

2.7 beamer slides (experimental)

A template is provided here on GitHub to produce HTML slides from `beamer`. The usage instructions are the same as for the main template (see Chapter 1).

The template contains a minimal working example with a few slides, which you can see rendered here in HTML. The sidebar with the list of slides can be toggled on and off using the “menu” button in the toolbar at the top of the page; you can also download the rendered PDF version of the slides by clicking on the PDF icon in the same toolbar.

The way it works is by setting the document class to `article` for the HTML version, and redefining the `frame` environment so that each frame becomes a subsection. Together with some appropriate CSS tweaks, we end up with an HTML document with one page per frame, which behaves like slides.

This uses a slightly different preamble than the main template (see the file `preamble/SoM_slides.tex`) in particular to avoid errors with `beamer` commands such as `\visible`, `\onslide`, etc. Furthermore, your `.tex` files are copied, and search-and-replace is performed to remove `beamer` overlay specifications (e.g. `<1-3>`, `<4>`, `<5->`), and to handle some of the possible ways to specify frame titles (see the `sed` commands in `.github/workflows/bookml.yaml` if you're not afraid of regular expressions).

There are some caveats, and it's not as robust as the workflow for producing course materials. The below is not an exhaustive list, and any suggestions for solutions or workarounds are very welcome – please post an issue on GitHub (or email `lt@maths.ed.ac.uk`).

- Any beamer settings or commands in your preamble must go in an `\iflaxml\else ... \fi` block, so that they don't cause errors when rendering HTML.
- If you already have `\subsection` commands in your slides, they will each produce their own separate slide.
- Frames without a title must begin with `\begin{frame}{}.`
- If you use `\frametitle`, it must be on the same line as `\begin{frame}.`
- Frames without a title show up as the title of your document in the sidebar.
- Multicolumn environments don't work (yet!).

We would recommend starting with editing the `main.tex` file in the template and replacing the frames with your own, rather than trying to convert one of your existing source files as it is.

3 Appendices

3.1 Context: why are we doing this?

The University is legally required to provide learning materials in accessible formats¹. Although guidance is provided through Information Services² for some document formats (e.g., MS Office), workflows deviating from these are not currently supported.

Particularly in subjects which require the extensive use of mathematical notation or other non-standard characters, the $\text{\LaTeX}$ typesetting language is widely used to generate PDF lecture notes, worksheets, and slides. However, these documents do not currently fulfil accessibility requirements (see Section 3.1.1). While there are various approaches to producing accessible documents from $\text{\LaTeX}$ source files, none of these are widely adopted across the University, or elsewhere. Barriers include technological expertise and means, workload, inconsistent quality of output, and lack of institutional and technical support.

The PTAS project “A systematic approach to enable production of accessible course materials” aimed to close this gap by identifying and overcoming such barriers in a systematic manner. Working in collaboration with a diverse community of educators and students, we have developed a set of guidance documents, technical instructions, and $\text{\LaTeX}$ templates to facilitate the creation of course materials in accessible, flexible output formats. We have particularly focused on HTML output, which is not only supported by a wide variety of devices, but can also be displayed using user-defined preferences, such as fonts, text sizes, colours, and wrapping for different screen sizes; it is also compatible with many screen readers.

3.1.1 Why are PDFs not “accessible”?

In our context, **accessibility** relates to course content being accessible (as in readable and usable) to everyone, in particular people with disabilities or health conditions. Written content can present accessibility barriers to different people. For example,

- those with a visual impairments may use assistive technology (e.g. screen readers) to access course materials;

¹Accessible and Inclusive Learning - University of Edinburgh

²Creating accessible materials - University of Edinburgh

- partially sighted or colourblind people, or those with difficulties related to processing text (for example people with dyslexia or ADHD), may need to customise fonts, font sizes, spacing, and colours to improve accessibility.

None of the above is practical to do with PDF files. In particular:

- PDF does not allow fonts, sizes or colours to be changed easily;
- $\text{\LaTeX}$ -produced PDF documents are not “tagged”³ because $\text{\LaTeX}$ does not (yet) support this feature⁴. A tagged PDF provides metadata on the document structure, which makes it possible for assistive technologies to navigate between sections — and therefore makes screen readers usable.
- Crucially, screen readers cannot read mathematical expressions inside a PDF file, because the PDF file does not contain any semantic information about the mathematical expressions — it only contains a graphical rendering of the expressions.

3.1.2 A solution: HTML with MathML

HTML documents are accessible in some ways that PDF documents are not.

- The user can easily change font size, font family, and colours⁵. It is also possible to print from HTML to paper while conserving your font settings – so you can print in large font, for example (which, again, is not straightforward with a PDF).
- The document structure and navigation relies on HTML tags, which are accessible by screen readers.
- Providing alternative text (or alt-text) for figures is easier. This is different from figure captions — it describes the key features of a figure under the assumption that the person reading the alt-text cannot see the image.
- Mathematical expressions are displayed using MathML⁶, which is compatible with a range of assistive technologies (e.g. screen readers and Braille displays)⁷.

MathML renders all mathematical expressions in a document, whether inline ($x = 1$) or in display mode:

$$f(x) = \int_0^\infty \left(\frac{e^{-t^2}}{\sqrt{\pi}} \right) \left(\sum_{n=0}^\infty \frac{(-1)^n x^{2n+1}}{(2n+1)!!} \right) dt \quad (3.1)$$

³The accessibility checker *Ally*, which is the tool embedded in Learn Ultra, determines the accessibility score of a PDF file largely depending on whether or not it is tagged.

⁴There is ongoing work to improve this at the $\text{\LaTeX}$ Project, but this is still in early stages; see the paper Enhancing $\text{\LaTeX}$ to automatically produce tagged and accessible PDF for a summary of progress as of March 2024.

⁵This can be done with custom style sheets (for instance via browser extensions such as Stylus), or Javascript (for example, try the toolbar at the top of the HTML version of this document).

⁶MathML homepage

⁷Brief explanation and demonstration of MathML, including a video showing use with a screen reader.

You may be familiar with MathJax⁸, used for instance on STACK. MathJax is a Javascript engine used to display MathML in the browser. This is a good primer on what they both do. In particular, MathJax offers a contextual menu on right-click with further accessibility features, including the possibility to copy the $\text{\LaTeX}$ source for an expression (try it with the expression above, in the HTML version of this document).

It is possible to generate HTML with MathML *automatically* from `.tex` source files — so we don't have to worry about doing any of this manually. A key part of the PTAS project was to review different workflows for doing so, in order to provide recommendations and appropriate support for robust, user-friendly production of accessible course materials.

3.1.3 Insights from staff interviews

During the Spring semester of 2023–24, we conducted interviews with teaching staff from the Schools of Mathematics, Physics and Astronomy, Engineering, and Philosophy, Psychology, and Language Sciences to better understand their current workflows in producing course materials. We specifically chose staff members who work extensively with $\text{\LaTeX}$.

Several common themes emerged from these discussions, which helped inform our approach:

Teaching staff have limited time. Any proposed adjustments must be integrable into existing workflows with absolutely minimal increase in workload. Solutions should prioritize simplicity, efficiency, and robustness, without the need to extensively consult documentation or complex instructions. Technical support should be provided to resolve problems.

Openness to adjustments in workflow. While the staff we interviewed expressed willingness to adjust their workflows to enhance accessibility of their materials, this may well be biased by us interviewing staff who put themselves forward to discuss this in the first place. This underscores the importance of making all recommendations user-friendly and low-friction to ensure widespread adoption.

Challenges of shared $\text{\LaTeX}$ documents. Documents in $\text{\LaTeX}$ authored by multiple contributors or handed down over time often exhibit inefficiencies such as redundant or inefficient use of packages. Additionally, they may contain large blocks of commented-out code from previous versions or authors, which can clutter the document and complicate maintenance. These remnants, while intended for reference or future use, often become outdated and may not serve future iterations of the document effectively. Such practices can hinder readability, increase file size, and make it challenging to navigate and update course materials efficiently.

⁸<https://www.mathjax.org/>

There are more similarities than differences. While a multitude of resources are employed in creating course materials, the majority of $\text{\LaTeX}$ documents typically share similar foundational structures. Variations typically arise from specialized packages tailored to specific subjects, use of different engines to compile PDFs, the number of authors involved, and other contextual factors. This implies that supporting solutions which accommodate different packages, custom macros, and figures/images will enable us to develop a solution applicable to a wide range of $\text{\LaTeX}$ documents.

3.2 Flexibility is the key

A driving principle we choose to adopt in this project is **flexibility**, both on the side of the users (usually students) and producers (usually lecturers) of course materials.

- Writers of course materials should be able (and appropriately supported) to produce accessible versions with as little change to their preferred workflow as possible. In any automatic conversion process, technical issues and incompatibilities are inevitable — but technical support, consultation, testing, and iterative refinement will drastically help to reduce this.
- Users should be able (and appropriately supported) to customise the display of course materials to a format that is most accessible to them. There are certainly general guidelines we can follow to produce accessible documents — but prescribing a specific format (e.g. prescribing a specific font family or colour palette) is not as good as giving people options.

Importantly: accessibility is about ensuring that **everyone** can engage fully. For instance, for many students, PDF course materials will always be their preferred option: they are useful for printing, annotating and note-taking on tablets, and can be easier to use off-line. We are very much **not asking lecturers to get rid of their PDF notes** unless they want to — the idea is to also provide alternative options.

3.3 Upload as ZIP

You can upload a .zip file directly to the Content Collection on your Learn page, as a “zip package”. Please note this method will not work if you upload the .zip file to your general content collection, rather than on the direct on the specific learn page.

This should only be done if there is no option to upload a SCORM version of the materials.

1. On your main Learn page, click the + button to add content, then select ‘Content Collection’ at the bottom.
2. Select ‘Browse content collection’.
3. At the top, click ‘Upload’, then ‘Upload zip package’.

4. Select 'Browse local files' and select the .zip file you have just created, or drag and drop the file into the box.
5. Click 'Submit'.
6. Click on the folder you have just uploaded, check the box next to `index.html` and click 'Submit'.
7. Click on the ... on the right to change the name from `index.html` to something more helpful for students (e.g. "Course notes"), and click 'Save'.

Important: with this method, you then need to add permission to the folder for students to view the whole site.

1. From the Learn home page, select "Tools", then "Content Collection".
2. Click on the folder corresponding to your course.
3. Hover on the folder containing your HTML course materials, click the arrow to get the contextual menu, and select "Permissions".
4. Click on "Select Specific Users by Place", then "Course".
5. Tick the name of your course, then tick "All course users" below.
6. Tick the "Read" option, leave the others unticked, and also leave "Overwrite" unticked.
7. Click "Submit".

Bibliography

- [PFCC16] Augustin Parret-Fréaud, Benjamin Cotté, and Antoine Chaigne, *Time-domain damping models in structural acoustics using digital filtering*, Mechanical Systems and Signal Processing **68** (2016), 587–607.
- [Str04] John C. Strikwerda, *Finite difference schemes and partial differential equations, second edition*, Society for Industrial and Applied Mathematics, 2004.